Parallel Matrix Multiplication with Java Threads: Speedup and Efficiency

Big Data - Assignment 3 - Degree in Data Science and Engineering, ULPGC, 2026-2027

Author: Pablo Mellado Martín

Github

Abstract

In this assignment I used the i-k-j multiplication from Assignment 2. Split the rows of matrix C across Java threads. I ran tests with 1, 2 3 4 6 8 and 12 threads to find out how faster the program ran. I used matrix sizes n = 500, 1000 and 2000. The greatest speedup was 6.6 for n = 1000 and 5.4 for n = 2000 both, with 12 threads. For n = 500 the speedup stayed below 3.2 because the multiplication finished in a few milliseconds so launching the threads added a lot of extra overhead. The improvement plateaued after 6 threads, which matches the number of cores. These results apply to this particular code and this laptop.

1. Introduction and objective

The question I wanted to answer is how time, speedup and efficiency change when adding threads to the i-k-j multiplication and at what point adding threads is no longer worth it.

Before measuring I expected the speedup to rise linearly up to six threads and then to increase more slowly, up to twelve threads because threads seven to twelve are logical threads that share a single core. I also expected matrices to scale less well because creating the threads costs time.

2. Background and references

In the i-k-j version each row of C can be calculated separately. It only needs to read A and B. Then write to that specific row. This means each row can be assigned to threads without any locks because no two threads write to the same location [1].

I used the metrics from the assignment, speedup S(p) equals T_seq divided by T_p and efficiency E(p) equals S(p) divided by p both calculated using medians. According to Amdahls law the part that cannot be run in parallel (in this case allocating C and starting the threads) sets a limit, on how much speedup can be achieved [2]. Another possible limit is memory: if all the threads read B at the time the memory bandwidth can become the problem [3].

3. Methodology

3.1 Implementations

The sequential version is the i-k-j method from Assignment 2 (multiplyIkj in Dense.java, which was already checked there). In this project it is called multiplySequential.

The parallel version works the same loop. The n rows of C are split into p blocks one, for each Java Thread and the main thread waits for all of them with join().

Both versions follow the algorithm use the same double[][]. The same input matrices and neither uses any library.

3.2 Inputs and environment

Item	Value
Matrices	A and B generated with `java.util.Random` (seeds 42 and 43), `double` values uniform in [0, 1)
Sizes	n = 500, 1000 and 2000
Threads	sequential, and 1, 2, 3, 4, 6, 8 and 12
CPU	AMD Ryzen 5 5625U, 6 physical cores, 12 logical (SMT), 16 MB L3, no affinity set
RAM and OS	16 GB DDR4-3200, Windows 11 Home 10.0.26200
Java	OpenJDK 21.0.12.1 (Temurin), HotSpot Server VM, `-Xmx4g`

I never used more than 12 threads, so the CPU was not oversubscribed. The JVM also runs some threads of its own (JIT compiler and garbage collector), but they do not take part in the multiplication.

3.3 Protocol and validation

The timer, which is System.nanoTime() records the time for allocating array C for creating and starting the threads for performing the multiplication and for waiting until every thread finishes. For each configuration I ran three up multiplications that were not counted. Then I ran five repetitions. I report the median and the IQR.

To check correctness I compared each repetition element by element with the result from the calculation. All 120 repetitions matched exactly. This is what should happen because each element is always computed by one thread in the order as, in the sequential code.

For memory I used the estimated size of the arrays $3 \times 8n^2$ bytes. This gives 5.7, 22.9 and 91.6 MiB. The memory size does not change with the number of threads because all threads share arrays A, B and C.

4. Results

Table 1. Median time (s), speedup S and efficiency E, 5 repetitions.

p	n = 500: time / S / E	n = 1000: time / S / E	n = 2000: time / S / E
sequential	0.0338 / 1 / -	0.458 / 1 / -	4.53 / 1 / -
1	0.0394 / 0.86 / 0.86	0.417 / 1.10 / 1.10	4.56 / 0.99 / 0.99
2	0.0215 / 1.57 / 0.78	0.221 / 2.08 / 1.04	2.94 / 1.54 / 0.77
3	0.0150 / 2.25 / 0.75	0.171 / 2.68 / 0.89	2.03 / 2.23 / 0.74
4	0.0120 / 2.82 / 0.70	0.132 / 3.47 / 0.87	1.52 / 2.98 / 0.75
6	0.0133 / 2.54 / 0.42	0.087 / 5.30 / 0.88	1.03 / 4.37 / 0.73
8	0.0107 / 3.15 / 0.39	0.071 / 6.43 / 0.80	0.87 / 5.23 / 0.65
12	0.0130 / 2.60 / 0.22	0.069 / 6.61 / 0.55	0.84 / 5.37 / 0.45

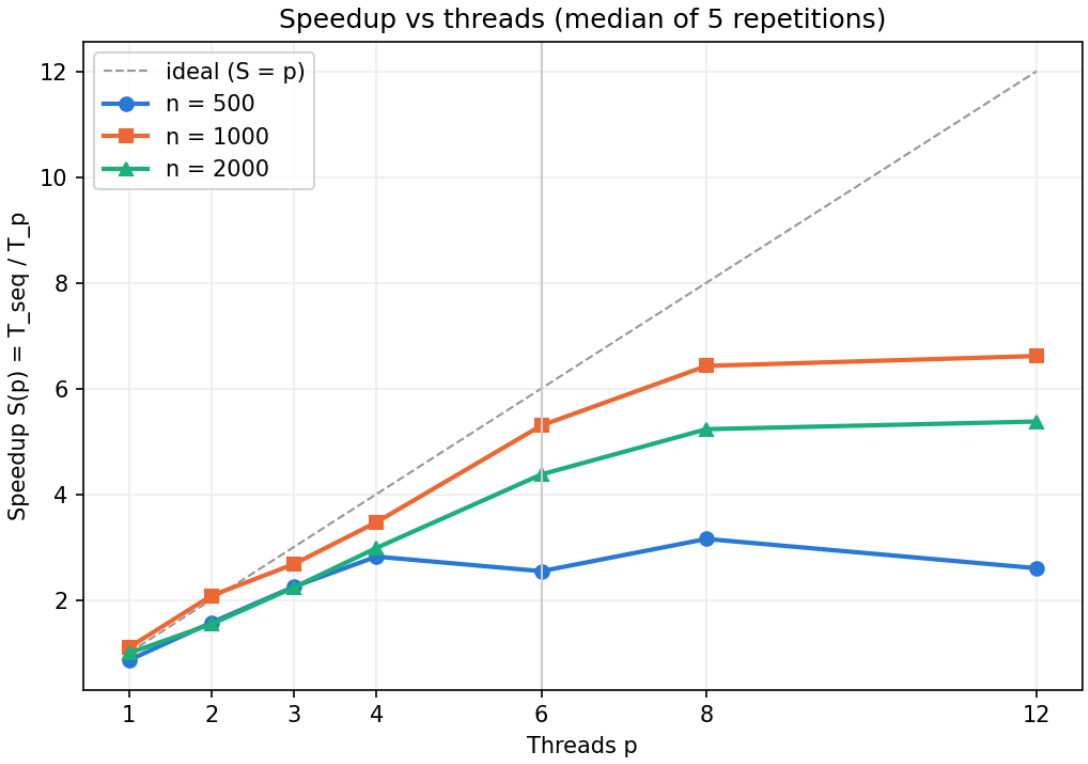


Figure 1. Speedup for each number of threads, with the ideal line S = p.

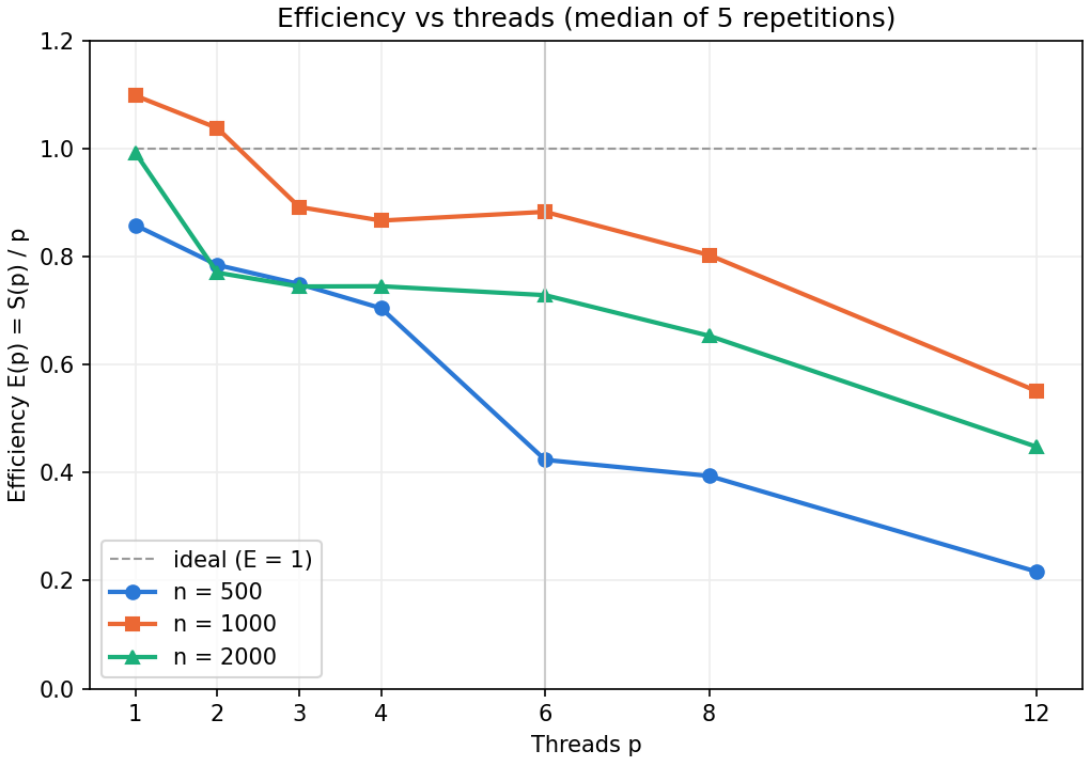


Figure 2. Efficiency for each number of threads.

5. Discussion

For n = 1000 and n = 2000 the speedup keeps increasing until we reach 6 threads. Between 3 and 6 threads the efficiency stays between 0.73 and 0.89. Going from 6 to 12 threads helps much less. For n = 1000 the speedup only goes from 5.3 to 6.6. For n = 2000 it goes from 4.4 to 5.4. I think this happens because threads 7 to 12 run on cores that're already busy. So they have to share the execution units and cache. This slows things down.

With n = 500 the situation is different. A single multiplication takes milliseconds. So the time spent creating and joining threads becomes very important. With one thread the parallel version is about 16 percent slower than the sequential version. That difference is mostly the cost of the thread itself. From 4 threads the speedup stops improving. Adding threads doesn't help.

For n = 2000 the efficiency starts at 0.77 with 2 threads. For n = 1000 it is close to 1. Matrix B is 32 MB in size. That is larger than the 16 MB L3 cache. So every thread has to read B from RAM at the time. This puts pressure on memory bandwidth, which limits the gain. I did not confirm this with hardware counters so it remains an explanation that fits the data.

For n = 1000 the efficiency with 1 and 2 threads is above 1—1.10 and 1.04. I do not think this is superlinear speedup. The parallel version with one thread does the same work as the sequential version.. It took 0.417 seconds instead of 0.458 seconds. The likely reason is that the JVM compiles the two methods differently. Also the sequential times varied quite a bit— range of

0.041 seconds. If I use the one-thread parallel version as the reference the efficiency, with 2 threads becomes 0.94.

There are some limitations. All tests were run on one laptop. The clock speed of that laptop can change. I only tried one way of splitting the work—blocks of rows. I did not fix thread affinity. I also did not do the vectorization part.

6. Conclusions

On this laptop, splitting the i-k-j multiplication by rows between Java threads gives correct results and a speedup of up to 6.6× (n = 1000). Most of the gain comes from the first 6 threads, one per physical core, and the logical threads add little. For small matrices the cost of the threads is the main limit, while for large matrices the limit is memory access. Things I did not test, and that could be studied next, are vectorisation and whether a different split or a reusable thread pool would reduce the fixed cost.